A Hardware Rendering of the Rho Calculus with Boolean Arithmetic

Design specification: source language, target machine, and verification guidance

F1R3FLY.io design note — draft 0.4

13 November 2026

Abstract

This note specifies a hardware rendering of the rho calculus extended with Boolean arithmetic. It fixes the source language (syntax, sorts, structural congruence, matching modulo the Boolean laws, and reduction), the canonical forms that make names comparable in constant time, and a machine that tiles the surface of an FPGA with small processors. Each processor owns a namespace given by a structural formula, reduces in the same clock cycle as every other, and exchanges processes with its neighbours over a mesh. The note also gives the organisation of the implementation in Bluespec and the obligations an implementation must discharge to count as verified. Decisions marked C are confirmed; those marked P are proposed by this draft. Draft 0.4 replaces the single sequential reactor of earlier drafts with the concurrent tile array; the changes are listed in Section 11.

Contents

1 Purpose and scope **2**

2 Decision register **3**

3 Source language **4**
3.1 Syntax . . . 4
3.2 Sorts . . . 4
3.3 Structural congruence . . . 5
3.4 Matching . . . 5
3.5 Reduction . . . 6
3.6 Replication . . . 6

4 Canonical forms **6**

5 Target machine **7**
5.1 Overview . . . 7
5.2 Namespaces and ownership . . . 8
5.3 The array and the mesh . . . 9
5.4 Words and nodes . . . 9
5.5 Heap, interner and reference counts . . . 10
5.6 Tables . . . 10
5.7 Matcher and mask solver . . . 11
5.8 Arbiter and entropy . . . 11
5.9 Concurrent reduction in a tile . . . 12
5.10 Instantiation . . . 13

5.11 Boundary . . . 13
5.12 Decision log and faults . . . 14
5.13 Parameters . . . 14

6 Tooling and code organisation **15**
6.1 Languages (**D9**) . . . 15
6.2 Rules for the Bluespec code . . . 15
6.3 Suggested module list . . . 15
6.4 Flow . . . 16

7 Compiler and image **16**

8 Verification **16**
8.1 What "verified" means . . . 16
8.2 Layers of evidence . . . 18
8.3 The step-refinement checker . . . 19
8.4 Invariants . . . 19
8.5 Directed test programs . . . 20

9 Staging **20**

10 Open questions and recorded limits **21**

11 Status and changes **21**

A Worked examples **22**

1 Purpose and scope

The goal is a machine whose instruction set *is* the calculus. Three things follow from taking that literally.

- The only state change is the COMM rule.
- Every source of nondeterminism in the calculus is a physical arbitration in the machine.
- Every independence in the calculus is physical concurrency in the machine. Reductions on channels in different namespaces happen in the same clock cycle, on different processors, with no scheduler between them.

The machine is therefore not one processor but a surface of them: the FPGA is tiled with identical processors, each assigned a namespace, and the running process is spread over the tiles according to the channels its components use.

The first target is an FPGA, described in Bluespec, which emits Verilog. The design is restricted to components expressible in synthesizable Verilog or VHDL for CMOS. Clockless and superconducting realisations are out of scope, but every interface between and within tiles is latency-insensitive, so a later self-timed version does not require a redesign.

In scope: the calculus of Section 3; the tile, the array and the mesh; a compiler from source text to a memory image; a verification plan. Out of scope for this draft: cost accounting, join patterns, `where` clauses and grading, the combinator instruction set, and arrays spanning more than one device. Hooks are left where these attach (Sections 5.8 and 9).

2 Decision register

ID	Decision	Status
D1	Boolean operands admit drops: $*x$ is a Boolean expression.	C
D2	The Boolean-algebra laws are part of structural congruence.	C
D3	Replication is derivable by the reflective encoding; a persistent receive is provided as sugar and implemented as a primitive.	C
D4	Ill-sorted terms are excluded by a basic spatial-behavioural type system, not by the machine.	C
D5	Distinct solutions of a Boolean pattern are distinct, observable races, on the same footing as COMM races.	C
D6	First target is an FPGA, via generated Verilog.	C
D7	A high-level functional hardware language is used in place of hand-written RTL.	C
D8	Each processor is a tuple-space machine: hash-consed term heap, send and receive tables, matcher, arbiter. Static elaboration of the Boolean-typed fragment is a later optimisation.	C
D9	The whole design is written in Bluespec (BH syntax). Clash versions of the pure datapath functions may follow. One Haskell reference library is shared with the compiler.	C
D10	The compiler is trusted for sorts. The machine keeps a sticky `FAULT_SORT` flag whose unreachability is a proof obligation. The flag is a debugging aid: it does not detect every ill-sorted program.	C
D11	Variables are a separate syntactic category V, instantiated in the machine as de Bruijn indices. Names remain quoted processes.	C
D12	Open Boolean code is canonicalised as a truth table over at most $K = 6$ variables; larger supports are rejected at compile time.	C
D13	Boolean patterns (other than a bare variable) match closed Boolean values only.	C
D14	Fairness is possibilistic: every enabled redex has nonzero probability at every step. Uniformity over redexes is not required.	C
D15	Entropy is an LFSR behind an interface; every arbitration is written to a decision log that makes runs replayable.	C
D16	The machine is board-agnostic: no vendor primitives, inferred block RAM only, all sizes are parameters.	C

ID	Decision	Status
D17	The machine is an array of identical processors (tiles) covering the FPGA, joined by a nearest-neighbour mesh.	C
D18	Each tile owns a namespace. Namespaces are given by structural formulae on the quoted process.	C
D19	A tile serves several channels and commits at most one COMM per clock cycle. The default array has about 100 tiles, each with about one hundredth of the device's memory.	C
D20	Reduction is concurrent. There is no global scheduler and no sequential reactor: tiles act in the same cycle, and inside a tile the stages are concurrent rules joined by queues.	C
D21	Namespace formulae are drawn from the fragment recognised by a deterministic bottom-up tree automaton. A name's class is computed when it is interned, and ownership is a table lookup on the class.	P
D22	Heaps are per tile. Only closed terms cross between tiles, as serialised packets that are re-interned on arrival. Static code is replicated in every tile.	P
D23	Routing is dimension-ordered with wormhole packets. A tile's inbound path never waits on its outbound path.	P
D24	Every clock edge is a boundary. Each tile keeps its own decision log, stamped with the cycle; the array's log is a partial order.	P

3 Source language

3.1 Syntax

Let V be a countable set of variables, ranged over by v, w. Processes P, Q, Boolean expressions B, names x, y, patterns p and deep patterns d are given by

$$
\begin{array}{rcl}
P, Q & ::= & \mathtt{0} \mid B \mid \mathtt{for}(p \leftarrow x)\, P \mid \mathtt{for}(p \Leftarrow x)\, P \mid x!(Q) \mid P \,|\, Q \mid *x \\
B & ::= & \mathtt{true} \mid \mathtt{false} \mid *x \mid {\sim}B \mid B \mathbin{\&\&} B \mid B \mathbin{||} B \mid B \mathbin{\mathtt{xor}} B \\
x, y & ::= & @P \mid v \\
p & ::= & v \mid @d \\
d & ::= & \mathtt{0} \mid \mathtt{true} \mid \mathtt{false} \mid v \mid p!(d) \mid \mathtt{for}(v \leftarrow p)\, d \mid {\sim}d \mid d \mathbin{\&\&} d \mid d \mathbin{||} d \mid d \mathbin{\mathtt{xor}} d
\end{array}
$$

Relative to the grammar that opened the session, **D1** adds $*x$ to B, **D3** adds the persistent receive $\mathtt{for}(p \Leftarrow x)\, P$, and **D11** separates variables from names. Quotation is written $@P$ and dereference $*x$ throughout. The production $*x$ is shared by P and B; which reading applies is settled by sorts.

A receive binds the variables of its pattern in its body. Every variable denotes a *name*. A variable in a deep-pattern position denotes the name of the matched subprocess, so matching $@(v \mathbin{\&\&} w)$ binds v and w to names of Booleans, and the body uses them as $*v$ and $*w$. A program is a closed process.

3.2 Sorts

There are two sorts of process, $\mathsf{Bool} \subseteq \mathsf{Proc}$, and correspondingly two sorts of variable: *Boolean-sorted* variables, which only ever denote $@\mathtt{true}$ or $@\mathtt{false}$, and *structural* variables, which may denote any name. A term is well sorted when every operand of a Boolean operator is of sort Bool; in particular $*v$ as an operand requires v to be Boolean-sorted, and a Boolean-sorted variable requires every message that can reach its receive to quote a Boolean.

The last condition is a property of channels, and is discharged by the spatial-behavioural type system (**D4**), which assigns a carried sort to each namespace. This note treats that system as a black box with the following interface.

- **Input.** A closed process.
- **Output.** Either a rejection, or a sort for every pattern variable, such that reduction preserves well-sortedness.

The reflective encoding of replication sends a quoted process on a channel that the process itself mentions, so the system needs a recursive formula to type it. Giving the persistent receive its own typing rule avoids that for programs that use only the sugar.

3.3 Structural congruence

Structural congruence $\equiv$ is the least congruence on processes containing:

1. alpha-equivalence of bound variables;
2. $(P \mid Q) \mid R \equiv P \mid (Q \mid R), \quad P \mid Q \equiv Q \mid P, \quad P \mid \mathtt{0} \equiv P$;
3. the laws of Boolean algebra on B, with `xor` defined by B_1 `xor` $B_2 \equiv (B_1 \mathbin{\&\&} {\sim}B_2) \mid\mid ({\sim}B_1 \mathbin{\&\&} B_2)$;
4. $*@P \equiv P$.

Name equivalence $\equiv_N$ is the least equivalence with $@P \equiv_N @Q$ whenever $P \equiv Q$, and $@{*x} \equiv_N x$. Law 4 is adopted as a convention; in the original presentation the same effect arises only through substitution.

By **D2** every closed Boolean expression is congruent to exactly one of `true` or `false`, so there are exactly two closed Boolean names up to $\equiv_N$. The requirement that substituting a Boolean name into $*y$ yields the *evaluation* of the Boolean is therefore a consequence of law 3 and not a separate rule: evaluation is normalisation.

3.4 Matching

A substitution σ maps variables to closed names and respects sorts.

Definition 1 (Match set)**.** For a pattern p and a closed name n,

$$\mathsf{match}(p, n) \;=\; \{\, \sigma \mid \mathrm{dom}(\sigma) = \mathrm{vars}(p),\ p\sigma \equiv_N n \,\}/{\equiv_N}$$

where substitutions are identified when they agree pointwise up to $\equiv_N$.

Matching is *modulo* $\equiv$. For structural patterns this is ordinary tree matching on normal forms, and the match set has at most one element. For Boolean patterns it is equation solving: $\mathsf{match}(@(v \mathbin{\&\&} w), @\mathtt{false})$ has three elements and $\mathsf{match}(@(v \mathbin{\&\&} w), @\mathtt{true})$ has one. Because Boolean-sorted variables range over two classes, a pattern with k Boolean variables has at most 2^k solutions.

Restrictions.

- (**D13**) A Boolean pattern built with an operator matches only a closed Boolean value. Matching an operator pattern against open Boolean code under a binder would be unification of Boolean functions and is excluded. A Boolean leaf, a constant leaf included, that meets a name not quoting a Boolean is a sort fault and not a failed match; so the pattern `@true` cannot be used to test whether a message is a Boolean.

- A pattern variable is a *mask variable* when it occurs under a Boolean operator in the pattern; every occurrence of it is then a Boolean leaf. Any other variable is bound structurally, whatever its sort, and its sort is checked where the body uses it. A pattern has at most K mask variables (**D12**).
- Deep patterns contain no parallel composition, as in the given grammar, so no matching modulo associativity and commutativity is needed.
- The receive pattern $\texttt{for}(v \leftarrow p)\, d$, which matches under a binder, is deferred; see Section 10.

3.5 Reduction

$$\frac{x \equiv_N y \qquad \sigma \in \mathsf{match}(p, @Q)}{\texttt{for}(p \leftarrow x)\, P \mid y!(Q) \;\rightarrow\; P\sigma} \;(\textsc{comm})$$

$$\frac{x \equiv_N y \qquad \sigma \in \mathsf{match}(p, @Q)}{\texttt{for}(p \Leftarrow x)\, P \mid y!(Q) \;\rightarrow\; P\sigma \mid \texttt{for}(p \Leftarrow x)\, P} \;(\textsc{comm}!)$$

$$\frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q} \;(\textsc{par}) \qquad \frac{P \equiv P' \quad P' \rightarrow Q' \quad Q' \equiv Q}{P \rightarrow Q} \;(\textsc{equiv})$$

There is no reduction under a receive or under a quote. Substitution $P\sigma$ replaces free variables by names and does not descend into closed quoted terms, which have no free variables.

A *redex* of a process is a triple (send, receive, solution). By **D5** two redexes that differ only in the solution are different transitions. A send whose channel has a receive but whose payload admits no solution is not a redex and stays in place.

3.6 Replication

With $D(x) = \texttt{for}(v \leftarrow x)\,(x!(*v) \mid *v)$, replication of P on a name x not otherwise used is $x!(D(x) \mid P) \mid D(x)$. The machine must run this term correctly without special support, which requires a general drop of quoted code. The persistent receive is the efficient form. Its specification is the encoding:

Obligation 1. For all p, x, P, the process $\texttt{for}(p \Leftarrow x)\, P$ is weakly bisimilar to its reflective encoding, under context-labelled transitions.

4 Canonical forms

The machine compares names by comparing machine words. That is sound only if every term is stored in a normal form that is unique up to $\equiv$. The normal form $\mathsf{nf}(P)$ is defined bottom-up.

Variables. Bound variables are replaced by de Bruijn indices: an occurrence is the number of variable binders between it and its own binder, counting each variable of a multi-variable pattern in left-to-right order of first occurrence. Closed subterms therefore have a representation that does not depend on their context, which is what permits sharing. This indexes V only; names are still quoted processes (**D11**).

Quote and drop. $*@P$ is replaced by $\mathsf{nf}(P)$, and $@*x$ by x.

Booleans. A Boolean expression whose operands are all closed is replaced by its value. Otherwise let $v_0 < \cdots < v_{k-1}$ be the variables it *depends on* (its support), ordered by index. The normal form is the pair of the support and the truth table $T \in \{0,1\}^{2^k}$, where bit $r = \sum_i b_i 2^i$ of T is the value of the expression at $v_i \mapsto b_i$. Variables on which the function does not depend are removed. A function that is the identity on a single variable v is stored as $*v$. If $k > K$ the program is rejected (**D12**).

Parallel composition. Compositions are flattened, `0` units are removed, and the components are sorted into ascending order of their machine words. Since each component is itself canonical and hash-consed (Section 5.5), word order is a total order on components up to $\equiv$. Duplicates are kept: composition is a multiset.

Other constructors. Sends, receives and persistent receives are normalised componentwise. Patterns are normalised by the compiler into pattern programs (Section 7); two receives are congruent only if their pattern programs are identical.

Obligation 2 (Canonicity)**.** For all well-sorted P, Q within the size bounds: $\mathsf{nf}(P) = \mathsf{nf}(Q)$ if and only if $P \equiv Q$.

Remark 1. The truth table over a minimal, ordered support is a canonical form for Boolean functions, so the Boolean part of Obligation 2 is immediate. The same 64-bit word serves three purposes: it is the normal form, it evaluates the function forwards (index it by the argument values), and it solves patterns backwards (Section 5.7).

5 Target machine

5.1 Overview

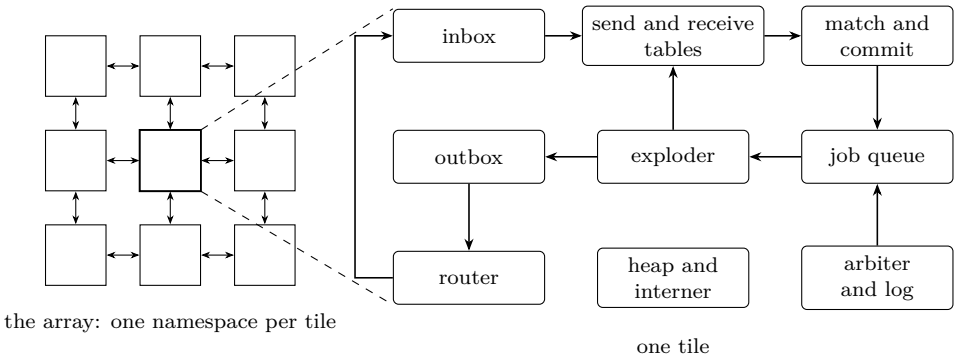


Figure 1: The array and one tile. Inside the tile, arrows show the way processes flow; each box acts in every cycle in which its inputs are ready.

The machine is a $W \times H$ array of identical *tiles*. A tile is a complete processor for the calculus, restricted to the channels of its own namespace. The running process is spread over the array as a *soup*: each top-level send or receive rests in the tile that owns its channel, or is in transit towards it. Everything quoted lives in the heap of the tile that holds it, as a hash-consed canonical term.

A COMM is a local event in one tile: the tile selects a redex among the sends and receives resting in it, removes them, and queues the instantiated continuation. The continuation is then taken apart, and each of its components is delivered to the tile that owns its channel, which may be the same tile or a distant one.

Tiles do not take turns. In one clock cycle every tile that holds a redex may commit one, and no tile needs to know what any other is doing, because redexes in different tiles share no send and no receive. Rho sends carry no continuation, so the time a component spends in transit is not observable, and the calculus needs no change to be distributed in this way.

5.2 Namespaces and ownership

Structural formulae. A namespace is a set of names, given by a formula on the process a name quotes. Formulae φ and name formulae α are

$$\begin{aligned} \varphi &::= \top \mid \texttt{0} \mid \texttt{true} \mid \texttt{false} \mid \alpha!(\varphi) \mid \texttt{for}(\alpha)\,\varphi \mid \Diamond\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \neg\varphi \\ \alpha &::= @\varphi \end{aligned}$$

They are read on closed normal forms. `0`, `true` and `false` hold of exactly those processes. $\alpha!(\varphi)$ holds of a send whose channel is in α and whose payload satisfies φ. $\texttt{for}(\alpha)\,\varphi$ holds of a receive, of either kind, on a channel in α whose body satisfies φ. $\Diamond\varphi$ holds when some parallel component satisfies φ; a process that is not a composition is its own single component. A term with a free variable satisfies no atom, so a formula sees into a receive's body only as far as the body is closed. There are no behavioural modalities and no counting of components: the formulae are structural, as **D18** requires, and satisfaction is invariant under $\equiv$ because it is defined on normal forms.

The namespace map. A map for an array of T tiles and P ports is a list of name formulae $\alpha_0, \ldots, \alpha_{T+P-1}$ that *partition* the closed names: every closed name satisfies exactly one. The tile or port t *owns* the names that satisfy α_t. The compiler checks the partition and rejects a map that fails it.

Realisation (D21). The compiler turns the map into one deterministic bottom-up tree automaton. Its states are called *classes*; there are at most 2^c of them.

- The class of a node is a function of its kind and the classes of its fields, given by small tables: $\delta_{send}(c_{chan}, c_{payload})$, $\delta_{recv}(c_{chan}, c_{body})$ and $\delta_{par}(c_{head}, c_{tail})$. Immediates have fixed classes, and so does every open term.
- δ_{par} is associative and commutative. It depends only on the set of component classes and on whether there is more than one component, which is what $\Diamond$ and the other atoms can observe.
- The class is computed when the node is interned and kept in the node header. The class of a name is the class of the process it quotes.
- A table τ of 2^c entries maps a class to its owner. Ownership of any name is therefore one lookup, from the name word and the local heap alone.
- The tables are identical in every tile, so a term has the same class wherever it is interned.

Partition is decidable for this fragment: disjointness is emptiness of an intersection and exhaustiveness is universality of a union, both on tree automata.

The default map. Symbols are numerals (Section 7), and "the numeral k with $k \equiv t \pmod{T}$" is regular: the class counts the depth of the numeral modulo T. The default map sends the numeral k to tile $k \bmod T$ and every other name to one designated tile. Under it, the compiler controls placement entirely by choosing which numeral each symbol receives. A program that mints channel names reflectively should supply its own map.

What ownership buys.

- **Races are local.** Two redexes compete only if they share a send or a receive, hence a channel, hence a tile. All arbitration is inside one tile.
- **Independence is physical.** Redexes in different tiles share nothing and commute, so they may fire in the same cycle.

- **Distance has a shape.** If the map comes from a tree of formulae, each tile's formula a conjunction along a path, then laying the tree out as an H-tree makes distance in the tree track distance on the mesh.

5.3 The array and the mesh

Geometry. Tiles are addressed by coordinates (x, y). Each has links to its four neighbours; a link is a pair of one-way channels with a valid/ready handshake, one flit wide. Ports sit beyond the edge of the mesh and are addressed as if they were tiles.

Packets (D22). A top-level send or receive travels as one packet.

- The head flit carries the destination, the kind (send or receive) and the length.
- The body is a *term stream*: the nodes of the term in post-order. Each field of a node is an immediate, a reference to an earlier node of the same packet, or a static address.
- The destination interns the nodes in arrival order. The stream is an interning script, and it preserves the sharing of the term.
- Only closed terms are sent. A receive closure bound for another tile is instantiated first and travels as a closed receive, arriving with an empty environment.

The static segment. The program's code, as compiled, is loaded at the same addresses in every tile's heap. It is read-only and never freed. A packet names a static node by its address, so program code, which is most of what moves, costs one flit however large it is.

Routing (D23). Routing is dimension-ordered: a packet travels along x to the destination column, then along y. It is wormhole switched: a packet holds each link from its head flit to its tail flit, and packets do not interleave on a link.

No deadlock. Dimension-ordered routing on a mesh cannot deadlock provided every destination consumes what arrives. The tile guarantees that: its inbound path (link, inbox, interner, tables) depends on no outbound resource. When the heap or a table is full the tile raises a resource fault, which halts the array; it never refuses a packet. Backpressure exists only on the forward path from commit to outbox, where a commit simply waits while the job queue is full.

Memory (D19). A device with M bits of block RAM and T tiles gives each tile M/T. The reference split of a tile's share is 60% heap nodes, 15% hash-cons table, 10% reference counts and class headers, 10% tables and environments, and 5% queues. The split is a parameter.

5.4 Words and nodes

All widths are parameters; the reference values are for a 32-bit word with a 28-bit heap address.

Word	Tag	Meaning
Process word	`ZERO`	the process `0`
	`BOOL` b	the Boolean value b
	`PTR` a	the process stored at heap address a
Name word	`QZERO`	the name `@0`
	`QBOOL` b	the name @b
	`QPTR` a	the name @(node at a)
	`VAR` i	the variable with de Bruijn index i (code only)

Closed Boolean names are immediates and never touch the heap.

Node kind	Fields
`PAR`	head: process word; tail: process word (a `PAR` or the last component)
`SEND`	chan: name word; payload: process word
`RECV`	chan: name word; pat: pattern-program address; body: process word; nvars; persistent bit
`DROP`	name: name word (always a `VAR`, by normalisation)
`BFN`	arity $k \leq K$; support: k variable indices, ascending; table: 2^K bits

Every node carries a header with its kind, a *free-variable depth*, and a *sort* bit (the node is of sort Bool). The depth of a node is the least k such that every free variable index in it is below k; a node is *closed* when its depth is zero. Both fields are computed at interning time from the children: a variable i has depth $i + 1$; a receive with n pattern variables has depth $\max(d_{chan}, d_{body} - n, 0)$; every other node takes the maximum over its fields. A single closed bit would not do: whether a receive is closed depends on whether its body's free variables are all bound by its own pattern, which a bit on the body cannot tell. The exact packing into block-RAM words is left to the implementation; it must be fixed in one shared definition used by the compiler, the reference model and the hardware.

Class. The header also carries the node's class (Section 5.2), c bits wide. Addresses are local to a tile: a `PTR` or `QPTR` word means nothing in another tile, which is why terms cross between tiles as streams and not as words. Static addresses are the exception, being the same everywhere.

5.5 Heap, interner and reference counts

- **Heap.** An array of nodes in block RAM with a free list.
- **Interner.** `intern(node)` returns the address of the unique stored node equal to its argument, allocating if there is none. It is a multi-cycle operation: hash the node contents, probe a hash table in block RAM, and confirm any hit by a full field comparison. Equality is exact; the hash only selects the bucket, so there is no collision risk.
- **Normalising constructors.** The interner is the only way to create a node, and each constructor normalises: `PAR` performs the sorted insertion of Section 4; `BFN` minimises the support and collapses constants and projections; `DROP` of a closed name returns the named process.
- **Reference counts.** Terms are acyclic, so reference counting reclaims everything. A count is held per node in a separate RAM. Decrements that reach zero push the node on a deferred-free stack that is drained by a background rule, which bounds the work per cycle.

Each tile has its own heap, interner and counts. The static segment (Section 5.3) occupies the same addresses in all of them.

5.6 Tables

Each tile has two tables, each with a valid bit and a busy bit per entry.

- **Send table** $S[0..N_S)$: channel (closed name word), payload (closed process word).
- **Receive table** $R[0..N_R)$: channel (closed name word), pattern-program address, body (process word, code), environment address, persistent bit.

Every channel in a tile's tables is owned by that tile.

A receive in the soup is a *closure*: its body is shared code and its free variables are supplied by an environment, a heap-allocated vector of closed name words. Anything that becomes a *name* is fully instantiated instead (Section 5.10), since names must be canonical to be compared.

Lookup by channel is a comparison of one name word against all entries of the other table. At the table sizes of a tile this is a register array with N_R comparators.

Top-level processes that are Boolean values are inert. Each tile counts the ones that come to rest in it in a two-counter *residue* register, so that they remain observable.

5.7 Matcher and mask solver

The compiler turns each pattern into a *pattern program*, a short sequence of operations executed against a name word with an explicit cursor stack.

Operation	Effect
`NBIND`	bind the current name word to the next structural variable
`NEQ` i	fail unless the current name word equals structural variable i
`QZERO`	fail unless the current name word is `QZERO`
`QSEND`	fail unless the current name quotes a `SEND`; push the quoted payload, then continue with the channel
`BLEAF` T	raise `FAULT_SORT` unless the current name word is `QBOOL` b; set $\mathit{mask} \leftarrow \mathit{mask} \wedge (b\ ?\ T : \neg T)$
`END`	succeed if the cursor stack is empty

All Boolean leaves of one pattern are expressed by the compiler over a single ordered space of that pattern's mask variables, so their masks conjoin directly. The mask starts as all ones. The result of a successful run is the vector of structural bindings and the mask.

- The mask is the solution set: bit r is set exactly when the assignment $v_i \mapsto$ bit i of r solves every Boolean leaf.
- The pair is a redex if and only if the run succeeds and the mask is nonzero.
- The solver returns $\mathit{count} = \mathrm{popcount}(\mathit{mask})$ and, given $u < \mathit{count}$ from the arbiter, the index r of the u-th set bit. An occurrence of a mask variable outside any operator is a leaf whose table is a projection, and a non-linear Boolean pattern such as $v \,\&\&\, {\sim}v$ needs no special case. A variable that occurs under no operator is not a mask variable and is bound by `NBIND`, as in the AND gate of Appendix A.

The pure parts (mask update, popcount, select) are combinational functions and are among the Clash components of Section 6.

5.8 Arbiter and entropy

All nondeterminism of the calculus passes through one module per tile, with one operation: `choose(n)` returns $u \in [0, n)$ for $n \geq 1$. A tile uses it three times in a cycle: to pick a send, to pick a receive among those on the same channel, and to pick a solution row.

- **No pruning.** For every n and every $u < n$ there must be an entropy value for which `choose(n)` returns u. A priority encoder is forbidden: it would make the machine a refinement of the calculus and not bisimilar to it.
- **Entropy.** Each tile has three 32-bit maximal-length Galois LFSRs with taps 32, 22, 2 and 1, one per choice, with distinct nonzero seeds that also differ from tile to tile. A choice among n takes the register's value w and returns $\lfloor w\, n / 2^{32} \rfloor$. Each register advances 32 positions per clock, which is one fixed layer of exclusive-ors, so that successive cycles use disjoint bits. A hardware random source may replace a register behind the same interface.

- **Fairness.** Only possibilistic fairness is required (**D14**). The three-stage choice does not induce the uniform distribution over redexes. When grading is added, each stage takes a weight vector in place of a count, and weighting a pair by its solution count makes the composite uniform; the `choose` interface is defined so that this is a change of argument type only.
- **Replay.** In replay mode the arbiter ignores entropy and takes its results from the tile's decision log.

5.9 Concurrent reduction in a tile

A tile is not a state machine that steps through phases. It is a set of rules, each guarded by the readiness of what it reads and writes, and all of them fire in every cycle in which their guards hold. They communicate through the tables and three queues: inbox, job queue and outbox.

Rule	Action
`ARRIVE`	Take flits from the inbox and intern them. When a packet is complete, insert its send or receive into the tables.
`COMMIT`	The COMM rule. Remove one send and one receive that form a redex, write the decision record, and queue the continuation as a job.
`EXPLODE`	Take a job apart. Deliver each component to its owner: into the local tables, or into the outbox as a packet.
`DEPART`	Move flits from the outbox to the router.
`ROUTE`	Forward flits between the four links, the inbox and the outbox.
`SWEEP`	Reclaim nodes whose reference count is zero, when the interner is free.
`SCAN`	Search the tables for any redex, for the quiescence report only.

COMMIT. In each cycle:

1. Choose uniformly among the valid sends that are not busy.
2. Compare its channel with every valid receive that is not busy, and choose uniformly among those that match.
3. Run the pattern program of that receive on the quoted payload.
4. If the run succeeds with a nonzero mask, choose a solution row. In the same cycle: remove the send; remove the receive unless it is persistent; write the decision record; and queue the job (body, environment). The environment is that of the receive, extended by the structural bindings and, for each mask variable, the `QBOOL` of its bit in the row. Variables enter in a fixed order: structural variables in order of first occurrence in the pattern program, then mask variables in order of mask position, so that the last mask variable is the most recently bound.
5. Otherwise nothing happens in this cycle. The next cycle chooses afresh.

A tile commits at most one COMM per cycle (**D19**). A redex that rests in the tile has a nonzero probability of being the one chosen in every cycle in which the job queue has room, which is the no-pruning requirement.

A pattern program that reads only immediates and the node its cursor starts on is evaluated combinationally, and the whole of COMMIT is one cycle. A deeper program is run by the matcher over several cycles. The send and the receive under examination are marked busy for the duration and are not offered to another match; the removal, the record and the job are still written in one cycle, when the match ends.

EXPLODE. A job is a body and an environment, and stands for the instantiated body. A work stack holds (process word, environment) pairs.

- `PAR`: push head and tail.
- `SEND`: resolve the channel through the environment, instantiate the payload (Section 5.10), and find the owner of the channel. If this tile owns it, insert into S; otherwise write the send to the outbox as a packet.
- `RECV`: resolve the channel and find its owner. If this tile owns it, insert a closure into R. Otherwise instantiate the receive and write it to the outbox.
- `DROP` of variable i: look up the name. `QZERO`: nothing. `QBOOL`: increment the residue. `QPTR`: push the named process with an empty environment. This is the general drop that the replication encoding needs; because the quoted term is closed and shared, it costs no copying.
- `BFN`: evaluate the table at the environment values; increment the residue.
- `BOOL`: increment the residue. `ZERO`: nothing.

A job is exploded over several cycles while the tile goes on arriving and committing.

Shared resources. `ARRIVE` and `EXPLODE` both use the interner and both insert into the tables. Where they conflict `ARRIVE` goes first, as **D23** requires. The order is not observable, because neither rule chooses a redex: each only moves a component from one place where it already counts as part of the process to another.

What a cycle means. Across the array, the `COMMIT`s that fire in one cycle are pairwise independent redexes, one per tile at most. The effect of the cycle on the process the array stands for is their joint reduction, in any order. Every other rule leaves that process unchanged.

Quiescence. A tile is *idle* when its job queue, inbox and outbox are empty and `SCAN` has completed a full pass over its (send, receive) pairs, finding no redex, since its tables last changed. The array is quiescent when every tile is idle and every link is empty; this is a registered conjunction over the array. `SCAN` is sequential, but it only reports. It never chooses.

5.10 Instantiation

`inst(code, env)` returns the canonical closed term obtained by substituting the environment into the code. It is a post-order walk with an explicit stack that rebuilds nodes through the interner, tracking the number k of binders passed.

- A subterm whose free-variable depth is at most k is returned unchanged: it mentions no variable beyond the binders already passed. This is the common case and makes instantiation proportional to the open part of the code only.
- `VAR` i with $i < k$ is local and unchanged; with $i \geq k$ it is replaced by environment entry $i - k$, where entries are numbered from the most recently bound. Environment entries are closed, so no index shifting is needed.
- A `BFN` whose support becomes partly known is cofactored on the known variables and re-normalised; when fully known it becomes a `BOOL`.

5.11 Boundary

The environment is modelled as processes on channels that the namespace map assigns to *ports*.

- **Egress.** A send or receive whose channel is owned by a port is routed to the edge of the mesh like any other packet and leaves on that port's valid/ready interface.
- **Ingress.** A port injects a packet at the edge tile it is attached to, addressed to the owner of its channel. In the first stages the payload is restricted to immediates, and the sort tag is checked against the port's declared sort; a mismatch is dropped and counted.

5.12 Decision log and faults

Per tile. Each tile writes one decision record per `COMMIT`: cycle number, send slot, receive slot, solution row. Further streams report every insert into S or R with its slot, every residue increment, and every packet that departs or arrives, with a packet identifier. Together these let an external checker reconstruct the soup without reading the machine's internals.

The array's log (D24). The array's log is the union of the tiles' logs. Records of the same cycle are unordered, and rightly so: they are independent. The log is therefore a partial order of reductions and not a sequence.

Replay. Given the outputs of its arbiters, the array is a deterministic synchronous circuit. Replaying each tile's log, keyed on the cycle number, reproduces every cycle of the run.

Faults are sticky status bits. A fault in any tile halts every tile on the following cycle.

Fault	Raised when
`FAULT_SORT`	a Boolean operand or Boolean leaf meets a non-Boolean name. Unreachable for well-sorted programs (**D10**).
`FAULT_HEAP`	the heap, hash table or environment space is exhausted
`FAULT_STORE`	a send or receive table is full
`FAULT_STACK`	a work stack overflows

Resource faults are the only way the machine departs from the calculus, and they are detected, never silent.

Remark 2 (The sort flag is not a sort check)*.* A Boolean law can erase an ill-sorted operand before anything inspects it: $*v$ `xor` $*v$ normalises to `false` and $*v$ && `true` to $*v$, whatever v is later bound to. `FAULT_SORT` therefore fires only for operands that survive normalisation. Its never firing does not show that a program is well sorted, and Obligation 8 cannot be turned round. The sort checker carries the whole obligation.

5.13 Parameters

Parameter	Stage 1 value	Meaning
$W \times H$	2×2, then 10×10	array dimensions
P	4	ports
c	6	class width; at most 64 classes
K	6	maximum Boolean support and pattern mask variables
N_S, N_R	32, 32	send and receive table sizes, per tile
A	16 (later)	heap address width, per tile; stage 1 has no dynamic heap
E	16	maximum environment length
D	16	work-stack depth
job queue	4	pending jobs per tile
inbox, outbox	16, 16	flits
flit	36 bits	one word and its framing
LFSR	3×32 bits	entropy state, per tile

6 Tooling and code organisation

6.1 Languages (D9)

Reference library (plain Haskell). Abstract syntax, normal forms, match sets, the reduction relation returning *all* redexes, the class automaton, and a model of the tile array. This is the executable semantics. The compiler and the checker are built on it.

Bluespec (BH). The whole of the hardware. The pure datapath (truth tables, masks, node packing, class tables) is written as BH functions in packages that hold no state, so that it can be tested on its own and later mirrored in Clash. Everything with state is a module of rules. The calculus is a rewrite system, and Bluespec's guarded atomic rules express the rules of Section 5.9 directly; its scheduler derives which of them may fire together.

6.2 Rules for the Bluespec code

1. Every semantically visible choice goes through the arbiter module. The Bluespec scheduler resolves rule conflicts by a fixed urgency order, which is a priority encoder; it must never be the thing that decides which redex fires.
2. Every rule is marked in a comment as *reducing* (it changes the process the array stands for, by one redex) or *administrative* (it does not). Only `mkCommit` contains reducing rules.
3. Any urgency annotation must carry a comment stating why the conflict it resolves is administrative.
4. No rule on the inbound path may have a guard that depends on outbound state (**D23**).
5. The tile is one module, instantiated $W \times H$ times with its coordinates and seeds as its only parameters. There is no tile-specific logic and no logic outside tiles other than the port adapters and the fault and quiescence trees.
6. Each rule that changes the tables or the heap states, as an assertion, the invariants of Section 8.4 it relies on.
7. No vendor primitives. Memories are inferred. One clock, synchronous reset.

6.3 Suggested module list

Package or module	Kind	Responsibility
`TruthTable`	pure	table algebra and normalisation
`MaskSolve`	pure	mask, popcount, select
`NodeCodec`	pure	packing, hash, free-variable depth and sort bit
`ClassTables`	pure	the class automaton and the owner table
`mkHeap`	rules	node RAM, static segment, free list, reference counts
`mkInterner`	rules	hash-cons table, normalising constructors, classes
`mkTables`	rules	send and receive tables, channel lookup, residue
`mkMatcher`	rules	pattern-program evaluation
`mkCommit`	rules	the COMM rule
`mkExploder`	rules	instantiate, explode, deliver
`mkPacketiser`	rules	term streams out and in
`mkRouter`	rules	dimension-ordered wormhole routing
`mkArbiter`	rules	`choose`, LFSRs, replay input
`mkLog`	rules	decision, insert, residue and packet streams
`mkTile`	rules	one tile
`mkArray`	rules	the mesh, ports, fault and quiescence trees

6.4 Flow

Source text → compiler → memory image. Bluespec → Verilog → simulation (Icarus Verilog, Verilator) and synthesis (vendor or open flow). The same Verilog is the input to bounded formal checks. The toolchain this draft was checked against is the Bluespec compiler 2026.07.1 with Icarus Verilog 13.

7 Compiler and image

The compiler is software and is in the trusted base for sorts only (**D10**); every other property of its output is checked dynamically by the verification harness.

1. Parse; resolve variables to de Bruijn indices.
2. Run the sort checker; record each pattern variable's sort.
3. Normalise (Section 4); reject Boolean supports above K.
4. Compile patterns to pattern programs. Assign mask variables their positions in the pattern's mask space and extend each leaf table to that space.
5. Intern all code into a heap image using the same interner algorithm as the hardware, so that addresses in the image are canonical.
6. Explode the top-level process into initial send and receive tables.
7. Emit block-RAM initialisation files for the heap, hash table, reference counts, pattern store and the two tables, plus a symbol file mapping source names to name words for the test harness.

Namespaces and placement. Three further steps serve the array.

1. Compile the namespace map to the class tables and the owner table, and check that it is a partition (Section 5.2).
2. Assign numerals to symbols. The default is order of first occurrence. The placing option chooses them to keep channels that exchange messages in the same tile or in neighbouring ones, since under the default map the numeral decides the tile.
3. Emit the static segment once, for every tile, and for each tile its initial tables: every initial component of the program is placed in the tile that owns its channel.

8 Verification

8.1 What "verified" means

Let s be the state of the array at a clock edge. The process it stands for is

$$\alpha(s) \;=\; \prod_{t}\Big(\prod_{i\in S_t} c_i!(q_i) \;\mid\; \prod_{j\in R_t} \mathsf{inst}(recv_j, env_j) \;\mid\; \prod_{k\in J_t} \mathsf{inst}(body_k, env_k) \;\mid\; \mathtt{true}^{\,t_t} \mid \mathtt{false}^{\,f_t}\Big) \;\mid\; \prod_{p} term(p)$$

where t ranges over tiles, J_t is the part of tile t's jobs not yet delivered, (t_t, f_t) is its residue, and p ranges over the packets between an exploder and a table, each standing for its send or receive however much of it has been transmitted. Every clock edge is a boundary (**D24**).

Obligation 3 (Soundness)**.** If the array goes from clock edge s to the next edge s', then $\alpha(s')$ is $\alpha(s)$ reduced by exactly the redexes whose `COMMIT`s fired in that cycle, and these are pairwise independent. If none fired, $\alpha(s') \equiv \alpha(s)$.

Obligation 4 (No pruning)**.** If $\alpha(s) \to Q$ by a redex whose send and receive rest in the tables of a tile whose job queue has room, that tile commits that redex in the next cycle with nonzero probability.

Obligation 5 (Delivery)**.** Absent faults, every component of every job and every packet comes to rest in the tables or residue of its owner within a bounded number of cycles. No packet is lost, duplicated or held for ever.

Obligation 6 (Quiescence)**.** The quiescent bit is set at a clock edge if and only if $\alpha(s)$ has no redex.

Obligation 7 (Replay)**.** Two runs from the same image with the same ingress sequence and the same per-tile decision logs pass through identical states at every clock edge.

Obligation 8 (Sort safety)**.** For a well-sorted image, `FAULT_SORT` is unreachable.

Obligation 9 (Placement independence)**.** For a fixed program, the processes reachable as $\alpha(s)$, up to $\equiv$, do not depend on the dimensions of the array, on the namespace map, or on the assignment of numerals to symbols.

Obligations 3, 4 and 5 together say that the fault-free array, observed at clock edges, is bisimilar to the calculus: the administrative moves are silent, and every redex of the calculus is, after a bounded delay, a redex some tile may commit. Obligation 9 says the tiling is an implementation matter and not a change of language. Obligations 1 and 2 are the supporting facts about the language itself.

8.2 Layers of evidence

Layer	Method	Discharges
V1	Property tests of the reference normaliser against a brute-force decision of $\equiv$ on small terms; exhaustive for Boolean code with $k \leq 3$	Ob. 2 (reference)
V2	The Clash functions are the reference functions: test in GHC, then check generated Verilog against them by random co-simulation	datapath correctness
V3	Interner test: intern random raw terms through the machine's normalising constructors alone; addresses equal iff reference normal forms equal, and readback equals the reference normal form	Ob. 2 (hardware)
V4	Step-refinement checker (below) on random and directed programs	Ob. 3, 6
V5	Formal cover on the arbiter: for every $n \leq 64$ and $u < n$ a witness entropy value exists	Ob. 4 (arbiter)
V6	Redex coverage: on small programs, every reference redex at every visited state is taken under some seed	Ob. 4 (system)
V7	Replay test: record, replay, compare boundary traces	Ob. 7
V8	Assertions for the invariants below, in simulation and bounded model checking	supports all
V9	Bounded bisimulation check, in the reference, of persistent receive against the encoding; paper proof to follow	Ob. 1
V10	Instantiation commutes with normalisation, in the reference and in the machine, against textbook substitution on raw terms	Ob. 3 (instantiation)
V11	Mutation: each of a set of bugs injected into the machine and into the reference must fail some layer	adequacy of V1–V10
V12	Placement independence: the same programs on 1×1, 2×2 and 10×10 arrays and under different maps reach the reference transition graph, administrative moves aside	Ob. 9
V13	Mesh: random traffic with every packet accounted for; bounded model checking of deadlock freedom on small meshes	Ob. 5
V14	Concurrency: runs in which several tiles commit in one cycle are produced and accepted, and commits per cycle are measured	**D20**

8.3 The step-refinement checker

The checker is a testbench component built on the reference library. It reads only the streams of Section 5.12, and the heap contents needed to decode the addresses they report.

1. It maintains the abstract state: a map from each tile's table slots to reference terms, the residue counts, the undelivered part of each job, and the packets in flight. This is α computed incrementally.
2. In each cycle it collects the decision records of all tiles. For each it checks that the triple is a redex of the abstract state: channels equivalent, the row in the reference match set. Records of one cycle come from different tiles and so use different sends and receives.
3. It applies those redexes together, turning each into a job that stands for the reference reduct.
4. It checks every administrative event for conservation: each component a job delivers is a component of that job's reduct, and is consumed from it; each packet that arrives equals one that departed, and is consumed from the flight set; nothing enters a table by any other way.
5. When the array reports quiescence it checks that the reference state has no redex.

Any discrepancy is a failure with the offending cycle and tile, which the replay logs reproduce exactly.

8.4 Invariants

I1. Every stored node is in normal form.

I2. No two distinct heap addresses hold equal nodes.

I3. Each reference count equals the number of references from nodes, tables, environments and stacks.

I4. Every channel word and every payload word in the store is closed.

I5. A `DROP` node's name is a `VAR`; a `BFN` has minimal, ascending support, is not constant and is not a projection.

I6. A persistent receive is never removed by a step.

I7. An entry marked busy is under examination by exactly one match in progress.

I8. The mask is nonzero at COMMIT, and the chosen row is a set bit.

I9. Exactly one decision record is written per `COMMIT`.

I10. Every channel in a tile's tables is owned by that tile.

I11. The static segment is identical in every tile and is never freed.

I12. A packet that departs arrives exactly once, at its destination. Flits of different packets do not interleave on a link.

I13. A tile writes at most one decision record per cycle.

I14. A tile's inbound path is never stalled while its heap and tables have room.

8.5 Directed test programs

Program	Checks
Forwarder, AND, XOR gates	forward evaluation; one solution per match
Half adder, full adder	composition of gates through channels
Inverse XOR (Appendix A)	two solutions, both reached; empirical frequencies
Inverse AND on `false`	three solutions, all reached
Unsatisfiable pattern v && $\sim v$ on `true`	no redex; send remains; quiescence
Two receivers, one send	COMM race; both outcomes reached
Replication by encoding	general drop; an unbounded run. Hash-consing shares every copy, so the heap does not grow: the run ends on the receive-table bound when the replicated process leaves receives behind, and otherwise never
Persistent receive	same traces as the encoding, fewer steps
Equal names, different syntax	@(`true` && `false`) and @`false` as channels are the same channel
Ill-sorted image (negative test)	`FAULT_SORT` is raised and halts
Forwarder across two tiles	a send and its receive start in different tiles; one packet; one COMM
Ping-pong between two tiles	sustained traffic in both directions; no deadlock; packet accounting
Independent gates in different tiles	commits in the same cycle; a partial-order log
One program on three array sizes	the same reference graph on each
A map that is not a partition (negative test)	rejected by the compiler

9 Staging

Stage	Content	Exit criterion
1. Boolean-core array	Names are symbols and Boolean immediates; shallow patterns; static code only, with no dynamic heap; fixed-size packets. All of the concurrency: tiles, mesh, the default namespace map, per-tile logs, quiescence. Arrays of 1×1, 2×2, then 10×10	V4 to V8 and V12 to V14 pass on gate and inverse-gate programs, in simulation and on an FPGA
2. Reflection	Per-tile heap, interner with classes, reference counts, instantiation, term-stream packets, general drop, general namespace maps	V1, V3, V10 pass; replication by encoding runs across tiles
3. Full patterns	`QSEND` and deeper patterns with the multi-cycle matcher; ingress of quoted terms	V9 passes; full directed suite
4. Typed fast path	Channels typed Boolean-only are elaborated to one-bit links and gates inside a tile; the tuple-space machine remains the fallback	elaborated and interpreted runs agree under the checker

Later attachments, not staged here: cost accounting as a debit at `COMMIT`, and rent on a

namespace as a charge on what rests in its tile; graded choice through the arbiter's weight interface; the combinator instruction set as an alternative code representation in the same heap.

10 Open questions and recorded limits

Open: matching under a binder. The pattern `for`$(v \leftarrow p)\, d$ matches a receive and its body. The body of the matched receive has a free bound variable, so a variable of d cannot simply bind "the name of the body". The candidate readings are: bind the body as an abstraction; or allow d to mention v and match up to renaming. The machine supports neither until this is settled, and the pattern is rejected by the compiler in the meantime.

Open: the formula fragment. **D21** takes "structural formula" to mean the fragment of Section 5.2: no behavioural modality, no counting of components, and nothing to say about open code. It was chosen because a tile can then decide ownership from a name alone, in one lookup. If namespaces in use need more than this, the class automaton is the thing that has to grow.

Open: skew. Nothing stops a map, or a program, from sending most of the traffic to one tile. A tile commits one COMM per cycle whatever its load, so a skewed map gives a sequential machine with a large idle border. The compiler's placing option addresses static programs only.

Recorded limits. These follow from confirmed decisions and are listed so that an implementer does not mistake them for oversights.

- **Board.** None is fixed (**D16**). Block-RAM budgets are set per board through the parameters.
- **Trust.** The compiler is trusted for sorts (**D10**). A self-defending chip would need a hardware loader and a type system checkable in time linear in the image.
- **Support bound.** $K = 6$ (**D12**). Lifting it needs reduced ordered decision diagrams in the interner.
- **Fairness.** Possibilistic only (**D14**). A prescribed distribution over redexes enters through the arbiter's weight interface.
- **Top-level Booleans.** Kept in the residue counters.
- **Operator patterns.** Closed Boolean values only (**D13**).

11 Status and changes

Status. A reference implementation, `rhoref` 0.1.0, exists: the executable semantics, a word-level model of *one* tile with a sequential step, and the step-refinement checker, in plain Haskell. It conforms to draft 0.3. Its semantics, normal forms, matcher and instantiation carry over unchanged; its machine model and checker must be extended to the array, the class automaton and the per-cycle obligations of this draft before any hardware is compared with it. No Bluespec has been written. Nothing has been proved; Obligation 1 in particular is tested and not yet proved.

Draft 0.4: the concurrent tile array.

1. **Tiling** (**D17**, **D19**, Sections 5, 5.3). The single machine is replaced by an array of identical tiles on a mesh, about a hundred by default, each with about a hundredth of the memory.

2. **Namespaces** (**D18**, **D21**, Section 5.2). Each tile owns a namespace given by a structural formula. The formulae are compiled to a class automaton that runs at interning time, so ownership is a table lookup.
3. **Concurrency** (**D20**, **D24**, Section 5.9). The sequential step is gone. Tiles commit in the same cycle; inside a tile the stages are concurrent rules; every clock edge is a boundary; the log is a partial order. Parallel reduction is no longer a late stage but the first.
4. **Distribution** (**D22**, **D23**). Heaps are per tile, terms cross as streams, static code is replicated, routing is dimension-ordered, and the inbound path never waits on the outbound.
5. **Language** (**D9**). The whole design is in Bluespec; Clash is optional and later.
6. **Verification**. Obligations are restated per cycle; delivery and placement independence are new; layers V12 to V14, invariants I10 to I14 and five directed programs are added.
7. **Entropy** is three registers per tile advancing 32 positions per clock, in place of one register advanced per choice.

Draft 0.3: corrections. Each was found by building or testing the reference, which implements the corrected reading.

1. **Free-variable depth** (Sections 5.4, 5.10). The node header carries a free-variable depth in place of a closed bit, which cannot be computed for a receive from its children.
2. **The sort flag is incomplete** (Section 5.12, **D10**). Boolean laws can erase an ill-sorted operand, so `FAULT_SORT` is a debugging aid and not a sort check.
3. **Mask variables** (Sections 3.4, 5.7, 5.9). Draft 0.2 said a bare Boolean-sorted variable is a projection leaf, while its appendix bound one with `NBIND`. The rule is now stated once: a variable is a mask variable exactly when it occurs under a Boolean operator in the pattern. The order in which variables enter the environment is fixed.
4. **A Boolean leaf on a non-Boolean** (Sections 3.4, 5.7) is a sort fault, not a failed match.
5. **Replication by encoding** (Section 8) ends on the receive-table bound, or never; it does not exhaust the heap.
6. **Entropy sampling** (Section 5.8) is specified: the LFSR polynomial, the map from a sample to a choice, and the reference model's 32 positions per choice.

The verification plan gains two layers. V10 was added because instantiation is where normal forms are re-established at run time. V11 was added because two injected bugs survived the first suite, which showed that the interner had been tested only on terms the reference had already normalised; V3 is restated accordingly.

A Worked examples

Let x, y, z, l, r be distinct closed names. Both examples are shown within one tile. When the names fall in different namespaces the same reductions happen, with sends and receives travelling to their owners first.

An AND gate

$$\texttt{for}(a \leftarrow x)\,\texttt{for}(b \leftarrow y)\,z!(*a \mathbin{\&\&} *b) \;\mid\; x!(\texttt{true}) \;\mid\; y!(\texttt{false})$$

- **Code.** Inside the inner body, b has index 0 and a has index 1. The payload is a `BFN` with support $(0, 1)$ and table $T = 1000_2$: only row $r = 3$, where both are true, is set.

- **Initial soup.** One receive on x with pattern program `NBIND; END` and an empty environment; sends $(x, \texttt{BOOL}\ 1)$ and $(y, \texttt{BOOL}\ 0)$.
- **Step 1.** The send on y has no receive, so the only redex is on x. The mask is all ones over zero Boolean variables, one solution. The environment becomes [@`true`]. Explode inserts a receive closure on y with that environment.
- **Step 2.** COMM on y. The environment becomes [@`true`, @`false`], so index 0 (b) is false and index 1 (a) is true: row $r = 2$. Instantiating the payload reads bit 2 of T, which is 0. Explode inserts the send $(z, \texttt{BOOL}\ 0)$.
- **Result.** $z!(\texttt{false})$, and the machine is quiescent.

An inverse XOR

$$\texttt{for}(@(p\ \texttt{xor}\ q) \leftarrow x)\ (l!(*p) \mid r!(*q))\ \mid\ x!(\texttt{true})$$

- **Pattern program.** `BLEAF` T; `END`, with p at mask position 0, q at position 1 and $T = 0110_2$.
- **Match.** The payload is `QBOOL` 1, so the mask is $T = 0110_2$: rows 1 and 2, count 2.
- **Choice.** The arbiter returns $u \in \{0, 1\}$. With $u = 0$ the row is 1: $p = \texttt{true}$, $q = \texttt{false}$. With $u = 1$ the row is 2: $p = \texttt{false}$, $q = \texttt{true}$.
- **Result.** Either $l!(\texttt{true}) \mid r!(\texttt{false})$ or $l!(\texttt{false}) \mid r!(\texttt{true})$. Both must be reachable, and the row is recorded in the log. Had the payload been `false`, the mask would be $\neg T = 1001_2$ and the two outcomes would be the equal pairs.

The second example is a one-bit entropy tap written in the calculus itself: the quality of the machine's random source is observable by a program.